DG INTERNS HUB

WEEK 8 – ADVANCED WEB APPLICATION INTERNSHIP

REST API Development & Advanced Backend Engineering

Project: DG Interns Hub – Job & Internship Portal

Prepared by: [Nishanth]

Department / Institution: [CSE / Nehru Institute of Enginnering and Te] **Submission date:** 14 October 2026

Template note: replace placeholders and insert genuine screenshots/test evidence before submission.

2. Week 8 Objective

Week 8 extends the Week 7 Job & Internship Portal by improving backend structure, security, reliability, database efficiency, and API usability. The aim is to make the application more maintainable and production-oriented.

Learning objectives

• Build professional REST APIs with consistent request and response formats.

• Separate routes, controllers, models, configuration, and middleware.

• Implement CRUD operations for jobs, users, and applications.

• Add request validation, authentication, authorization, and centralized error handling.

• Support job search, combined filtering, pagination, and sorting.

• Protect endpoints and sensitive configuration using security practices.

• Test API behavior with Postman and document endpoints for frontend integration.

• Improve MongoDB schema design, references, constraints, and indexes.

Project scope

The portal supports internship discovery, user authentication, job details, and internship applications. Week 8 focuses on the backend APIs behind those features. Completion should be demonstrated with source code, API responses, database records, and test results.

3. Backend Architecture

The recommended architecture separates responsibilities so each module is easier to test and maintain. The server entry point should configure middleware and mount routes rather than contain all business logic.

Recommended folder structure

```
week-8-backend/
■■■ server/
■ ■■■ config/database.js
■ ■■■ controllers/{auth,job,application,user}Controller.js
■ ■■■ models/{User,Job,Application}.js
■ ■■■ routes/{auth,job,application,user}Routes.js
■ ■■■ middleware/{auth,role,error,validation}Middleware.js
■ ■■■ validators/
■ ■■■ utils/helpers.js
■ ■■■ server.js
■■■ postman/DG-Interns-Hub-API.json
■■■ API-DOCUMENTATION.md
■■■ README.md
■■■ .gitignore
■■■ package.json
```

Responsibilities

• **Config:** environment-based settings and MongoDB connection.

• **Models:** fields, validation, references, and indexes.

• **Controllers:** business logic and response handling.

• **Routes:** map HTTP methods and paths to controllers.

• **Middleware:** token verification, role checks, validation, and error handling.

• **Validators/utilities:** reusable validation rules and helper functions.

Align this structure with the actual codebase and include a screenshot of the real folder structure.

4. REST API Design

The API is organized by resource. HTTP methods communicate operations, while status codes and JSON response bodies indicate results. The table below is the Week 8 target specification; verify each endpoint against the running backend.

Required endpoint summary

Resource	Method & endpoint	Purpose
Auth	POST /api/auth/register	Create account
Auth	POST /api/auth/login	Authenticate user
Auth	GET /api/auth/me	Get current user
Auth	POST /api/auth/logout	Logout / invalidate session if supported
Users	GET /api/users/profile	Read own profile
Users	PUT /api/users/profile	Update own profile
Users	DELETE /api/users/profile	Delete own account
Jobs	GET /api/jobs	List/search/filter/paginate/sort
Jobs	GET /api/jobs/:id	Read one job
Jobs	POST /api/jobs	Create job; authorized role required
Jobs	PUT /api/jobs/:id	Update job; authorized role required
Jobs	DELETE /api/jobs/:id	Delete job; authorized role required
Applications	POST /api/applications	Apply for job
Applications	GET /api/applications/my	List own applications
Applications	GET /api/applications/:id	Read authorized application
Applications	PUT /api/applications/:id/status	Update status; authorized role required
Applications	DELETE /api/applications/:id	Delete authorized application

Standard response format

```
Success: {"success": true, "message": "Operation successful", "data": {}}
Error: {"success": false, "message": "Safe user-readable explanation"}
```

Document endpoints as operational only after testing. Logout must be described accurately based on whether the backend invalidates tokens or the client merely clears its session.

5. Authentication & Authorization

Authentication establishes who the user is. Authorization determines which resources and operations the user may access. The Week 8 design uses password hashing and JWT-based authentication for protected routes.

Authentication flow

• Registration validates fields and checks whether the email is already registered.

• Password is hashed using bcrypt before storage.

• Login verifies the submitted password against the stored hash.

• The server issues a signed JWT with an expiry after successful authentication.

• The client sends the token in the Authorization header as a Bearer token.

• Authentication middleware verifies the token before allowing protected requests.

Role-based access

Use role middleware to distinguish candidates, recruiters, and administrators where those roles exist. Candidates should only access their own profile and applications. Job management and application-status updates should be restricted to the correct role.

Evidence to include

Insert screenshots of successful registration/login, an authenticated request, missing/invalid token response, and a wrong-role request. Hide passwords and live tokens.

6. Security Implementation

The brief requires six baseline practices and at least two additional protections. The following table describes the expected controls; report only those implemented and verified in the actual code.

Practice	Implementation goal
JWT authentication	Verify signed tokens and enforce expiry.
Password hashing	Store hashes, never plaintext passwords.
Input validation	Validate required fields, formats, lengths, IDs, allowed values.
Protected routes	Require authentication for private operations.
Role authorization	Check permissions before privileged operations.
Environment variables	Keep DB URIs and signing secrets outside source control.
Rate limiting	Limit repeated requests to reduce abuse.
Helmet	Set protective HTTP headers.
CORS	Allow only intended frontend origins.
Body-size limits	Reject excessively large request bodies.
MongoDB query protection	Validate/sanitize query inputs.
Safe errors	Avoid stack traces and secrets in responses.

Attach configuration screenshots with secret values hidden. Never commit .env, passwords, database URIs, or live tokens.

7. Database Design & Optimization

MongoDB stores portal entities as documents. Mongoose schemas provide structure, validation, references, and index definitions. Query design helps the application remain responsive as the number of internships grows.

Suggested collections

• **User:** name, normalized unique email, password hash, role, timestamps.

• **Job:** title, company, location, type, category, skills, description, stipend, duration, deadline, timestamps.

• **Application:** user reference, job reference or saved job details, application fields, status, timestamps.

Optimization checklist

• Use a unique index for normalized user email.

• Index fields used frequently for filtering and sorting based on actual query patterns.

• Use references between users, jobs, and applications where appropriate.

• Perform filtering, sorting, pagination, and counts in MongoDB rather than downloading every record into React.

• Prevent duplicate applications using a suitable uniqueness strategy.

• Validate ObjectId values and return safe not-found responses.

• Use projections to avoid returning sensitive or unnecessary fields.

Pagination formula

```
skip = (page − 1) × limit
totalPages = ceiling(total ÷ limit)
```

Include a screenshot of MongoDB collections and index configuration without exposing connection strings.

8. Postman API Testing

The project owner has reported that Postman testing is completed. Include the actual collection and test-run evidence in the final submission; this report does not independently verify test execution.

Recommended test coverage

Area	Tests
Authentication	Register, login, invalid login, duplicate registration, current-user endpoint, logout behavior
Jobs	Create, list, read one, update, delete
Search & discovery	Search title/company/skills/location; combined filters; pagination; newest/oldest/deadline sorting
Applications	Apply, list own applications, read authorized application, update status with correct role, delete
Security	Missing/invalid/expired token, wrong role, invalid input, inaccessible resource
Error handling	Invalid ID, missing resource, malformed body, unexpected server error

Insert genuine screenshots

1. Authentication response
2. Job list and pagination metadata
3. Search/filter response
4. Application creation and retrieval
5. Validation or authorization error

Do not fabricate test results. Screenshots should match the requests in DG-Interns-Hub-API.json.

9. API Screenshots & Submission Evidence

Replace each placeholder with a genuine screenshot from the running application. Screenshots should prove the feature and avoid exposing secrets.

A. Backend folder structure

Show config, controllers, models, routes, middleware, validators, utils, and server.js.

[INSERT VERIFIED SCREENSHOT HERE]

B. Authentication API

Show a successful login/register response with secrets hidden.

[INSERT VERIFIED SCREENSHOT HERE]

C. Job APIs

Show the jobs endpoint returning multiple records.

[INSERT VERIFIED SCREENSHOT HERE]

D. Search, filters, and pagination

Show query parameters and response metadata.

[INSERT VERIFIED SCREENSHOT HERE]

E. Postman collection

Show imported collection and test results.

[INSERT VERIFIED SCREENSHOT HERE]

F. Error response

Show a safe validation or authorization error.

[INSERT VERIFIED SCREENSHOT HERE]

G. MongoDB

Show collections and sample records without connection strings.

[INSERT VERIFIED SCREENSHOT HERE]

H. GitHub repository

Show source, README, API docs and Postman collection; ensure .env is excluded.

[INSERT VERIFIED SCREENSHOT HERE]

10. Learning Outcomes & Conclusion

Week 8 provides practical experience in building a structured backend for a multi-role job and internship portal. The intended outcome is an API that is easier to maintain, test, secure, and integrate with the React frontend.

Expected learning outcomes

• Understanding REST resource design and HTTP methods.

• Separating business logic into controllers, models, routes, and middleware.

• Implementing authentication and role-based authorization.

• Validating requests and returning consistent error responses.

• Using search, filtering, pagination, and sorting in database queries.

• Testing APIs with Postman and documenting endpoint behavior.

• Applying database indexes and references where appropriate.

• Preparing source code and evidence for professional submission.

Conclusion

DG Interns Hub is intended to evolve from the Week 7 portal into a more structured and secure backend application. Before submission, verify every requirement against the running code, complete missing documentation, and replace placeholders with genuine screenshots and test evidence.

Prepared by: [Your Name] | Project: DG Interns Hub | Week 8